Duoc UC | ONLINE



DESARROLLO BACKEND III

Informe técnico

José Ignacio Soto Southerland

Contenido

1. Introducción....3

2. Objetivos....4

Objetivo General....4

Objetivos Específicos....4

3. Marco Teórico....5

3.1 Arquitectura de Microservicios....5

3.2 Spring Batch....5

3.3 Patrón Backend for Frontend (BFF)....5

3.4 Spring Cloud....6

3.5 Apache Kafka....6

3.6 OAuth2 y JWT....6

4. Arquitectura de la Solución....7

4.1 Visión General....7

4.2 Diagrama de Arquitectura....8

4.3 Flujo de una Petición Típica....8

4.4 Despliegue Híbrido: Local + AWS....8

5. Desarrollo de la Solución....9

5.1 Migración de Procesos Batch....9

5.2 Patrón Backend for Frontend....10

5.3 Microservicios con Spring Cloud....11

5.4 Seguridad Distribuida....12

5.5 Mensajería Asíncrona....13

5.6 Despliegue en la Nube....13

6. Resultados Obtenidos....14

6.1 Procesos Batch — Comparación de Paralelismo....14
6.2 BFFs — Optimización por Canal....15
6.3 Resilience4j — Circuit Breaker....16
6.4 Mensajería Kafka....16
6.5 Despliegue en AWS....17
7. Análisis Comparativo: Legacy vs Nueva Arquitectura....17
8. Desafíos y Soluciones....18
Desafío 1: Configuración de Listeners de Kafka en AWS....18
Desafío 2: Firma del Fallback en Resilience4j....18
Desafío 3: Datos Sucios en CSV Legacy....19
Desafío 4: Consistencia de Datos en el Batch Paralelo....19
Desafío 5: Limitaciones de Memoria en el Entorno de Desarrollo....19
Desafío 6: Migración a PostgreSQL para Batch....20
9. Conclusiones....20
10. Propuestas de Mejora....21
11. Bibliografía....22
12. Anexos....23

1. Introducción

El Banco XYZ es una institución financiera con más de 30 años de trayectoria en el mercado. Su sistema de gestión bancaria ha operado históricamente sobre una plataforma legacy basada en COBOL y scripts Shell, ejecutándose sobre servidores mainframe. Esta infraestructura maneja operaciones críticas como el procesamiento de transacciones, la generación de estados de cuenta, el cálculo de intereses y la gestión de clientes, siendo el pilar tecnológico sobre el cual se han sustentado décadas de operación bancaria ininterrumpida.

Con el paso del tiempo, el sistema legacy ha presentado limitaciones significativas que han comprometido su capacidad de adaptación a las exigencias del mercado actual. En primer lugar, la escalabilidad se encuentra restringida porque el mainframe impone límites físicos y lógicos difíciles de superar sin inversiones millonarias en hardware especializado. En segundo lugar, los costos de mantenimiento se han incrementado sostenidamente debido a que el talento especializado en COBOL es cada vez más escaso y costoso en el mercado laboral. Adicionalmente, existe una dificultad creciente para integrar el sistema con tecnologías modernas, ya que los protocolos legacy no se alinean con los estándares actuales de comunicación entre servicios. Los tiempos de despliegue también representan un obstáculo importante, pues cada cambio en el sistema requiere paradas programadas que afectan la disponibilidad del servicio. Finalmente, la rigidez arquitectónica del monolito implica que un fallo en un módulo específico puede propagarse y afectar el funcionamiento de todo el sistema, comprometiendo la continuidad operativa del banco.

Para superar estas limitaciones estructurales, el banco tomó la decisión estratégica de modernizar su infraestructura tecnológica migrando hacia una arquitectura de microservicios alojada en la nube. Este enfoque permite descomponer el sistema monolítico en servicios independientes, cada uno con su propio ciclo de vida, escalabilidad y tolerancia a fallos. El presente informe documenta el proceso completo de migración, abarcando cuatro ejes fundamentales: la reescritura de los procesos batch legacy utilizando Spring Batch, la implementación del patrón Backend for Frontend (BFF) para optimizar las respuestas según cada canal de cliente, el desarrollo de microservicios resilientes con Spring Cloud y Resilience4j, y el despliegue de la infraestructura de mensajería asíncrona en Amazon Web Services (AWS) mediante instancias EC2

2. Objetivos

Objetivo General

Diseñar e implementar una arquitectura moderna de microservicios para el Banco XYZ que sustituya los procesos legacy, garantizando escalabilidad, resiliencia, seguridad y capacidad de adaptación a las necesidades actuales del mercado.

Objetivos Específicos

1. Migrar los tres procesos batch críticos (Reporte de Transacciones Diarias, Cálculo de Intereses Mensuales y Generación de Estados de Cuenta Anuales) a Spring Batch, aplicando manejo avanzado de errores, paralelismo y escalabilidad.
2. Implementar el patrón Backend for Frontend para tres canales diferenciados (Web, Móvil y Cajeros Automáticos), optimizando las respuestas según las necesidades específicas de cada plataforma.
3. Desarrollar microservicios de negocio (Transacciones, Cuentas, Pagos y Clientes) con alta resiliencia mediante Spring Cloud y Resilience4j.
4. Integrar Apache Kafka como sistema de mensajería asíncrona para desacoplar productores y consumidores de eventos.
5. Desplegar la infraestructura de mensajería en AWS EC2, demostrando la capacidad de operar en un entorno cloud real.
6. Asegurar la comunicación entre servicios mediante OAuth2 y JWT firmados con RSA.

3. Marco Teórico

3.1 Arquitectura de Microservicios

La arquitectura de microservicios descompone una aplicación monolítica en servicios pequeños, independientes y desplegables de forma autónoma. Cada microservicio:

- Encapsula una capacidad de negocio específica
- Tiene su propio ciclo de vida de desarrollo y despliegue
- Se comunica con otros servicios mediante protocolos ligeros (HTTP/REST, mensajería)
- Puede escalar horizontalmente según demanda

3.2 Spring Batch

Spring Batch es un framework para el procesamiento de grandes volúmenes de datos. Sus componentes principales son:

- Job: unidad de trabajo completa
- Step: fase de un job (lectura → procesamiento → escritura)
- ItemReader / ItemProcessor / ItemWriter: responsables de leer, transformar y escribir datos
- JobRepository: persiste metadatos de ejecución
- TaskExecutor: permite el procesamiento paralelo

3.3 Patrón Backend for Frontend (BFF)

El patrón BFF propone crear un backend específico para cada tipo de cliente (Web, Móvil, ATM), evitando que todos consuman el mismo backend genérico.
Ventajas:

- Optimización de respuestas según las necesidades del canal
- Reducción de tráfico en clientes con ancho de banda limitado
- Seguridad diferenciada por canal
- Autonomía de equipos frontend y backend

3.4 Spring Cloud

Conjunto de herramientas para construir sistemas distribuidos:

- Config Server: configuración centralizada
- Netflix Eureka: descubrimiento de servicios
- Resilience4j: tolerancia a fallos (Circuit Breaker, Retry, Rate Limiter)
- LoadBalancer: balanceo de carga en llamadas entre servicios

3.5 Apache Kafka

Plataforma distribuida de streaming de eventos con las siguientes características:

- Alto throughput: millones de mensajes por segundo
- Persistencia: los mensajes se almacenan en disco
- Particionamiento: permite procesamiento paralelo
- Desacoplamiento: productores y consumidores independientes
- Consumer Groups: escalabilidad horizontal de consumidores

3.6 OAuth2 y JWT

OAuth2 es un framework de autorización que permite a las aplicaciones obtener acceso limitado a recursos. Los tokens JWT (JSON Web Tokens) son el formato estándar para transmitir claims entre servicios. Se firman con claves RSA de 2048 bits para garantizar autenticidad.

4. Arquitectura de la Solución

4.1 Visión General

La solución implementada se compone de 14 contenedores Docker organizados en cuatro capas:

Capa de Infraestructura:
- Config Server (8888): configuración centralizada
- Eureka Server (8761): descubrimiento de servicios
- Auth Server (9000): OAuth2 Authorization Server
- PostgreSQL (5433): base de datos persistente
- Kafka + Zookeeper: mensajería asíncrona

Capa de Microservicios de Negocio:
- microserviciotransacciones (8081): Kafka Producer
- microserviciocuentas (8082): Kafka Consumer
- microserviciopagos (8085): Kafka Producer
- microservicioclientes (8086): CRUD de clientes

Capa de Backend For Frontend:
- bffweb (8080): 6 campos por transacción
- bffmovil (8083): 4 campos por transacción
- bffatm (8084): 2 campos + operación de retiro

Capa de Procesamiento Batch:
- batchservice (8087): 3 jobs con paralelismo configurable

4.2 Diagrama de Arquitectura

Este se encuentra en el anexo A.

4.3 Flujo de una Petición Típica

1. El cliente solicita un token OAuth2 al Auth Server
2. Envía la petición al BFF con el token en el header ‘Authorization’
3. El BFF valida el JWT contra la configuración OIDC
4. El BFF propaga el mismo token al microservicio vía RestClient
5. El microservicio valida el token, consulta su BD y responde
6. El BFF aplica transformaciones específicas del canal
7. Devuelve la respuesta optimizada al cliente

4.4 Despliegue Híbrido: Local + AWS

La arquitectura final combina:

- Local (Docker Desktop): todos los servicios excepto Kafka
- AWS EC2: Apache Kafka + Zookeeper

Los microservicios locales se conectan al broker remoto mediante el parámetro ‘SPRING_KAFKA_BOOTSTRAP_SERVERS=<IP_PUBLICA_EC2>:9092’. Esta configuración permite aprovechar la nube sin requerir una migración completa de toda la infraestructura.

5. Desarrollo de la Solución

5.1 Migración de Procesos Batch

Objetivo: Reescribir los tres procesos legacy (COBOL/Shell) en Spring Batch 5.1.2.

Procesos migrados:

a) Reporte de Transacciones Diarias

- Función: Procesar transacciones del día, detectar anomalías y generar resúmenes
- Job: ‘reporteTransaccionesDiariasJob’
- Procesamiento: secuencial con skip policies
- Validaciones: fecha en 4 formatos, tipo (CREDITO/DEBITO), monto positivo y dentro de rango
- Salida: tabla ‘transacciones’ en PostgreSQL

b) Cálculo de Intereses Mensuales

- Función: Aplicar intereses sobre cuentas de ahorro y préstamos
- Job: ‘calculoInteresesMensualesJob’
- Tasas aplicadas: AHORRO 3%, PRÉSTAMO 12%, HIPOTECA 6%
- Cálculo: saldo × (tasa anual / 12)
- Salida: tabla ‘intereses’ en PostgreSQL

c) Generación de Estados de Cuenta Anuales

- Función: Compilar datos anuales por cuenta para auditorías
- Job: ‘generacionEstadosCuentaAnualesJob’
- Paralelismo: configurable (1, 2, 4, 8 hilos)
- Particionamiento: el CSV se divide en N rangos de líneas procesados en paralelo
- Salida: tabla ‘estados_cuenta’ en PostgreSQL

Mecanismos de tolerancia a fallos:

- Retry: reintentos para excepciones de acceso transitorio (‘TransientDataAccessException’)
- Skip: filtrado silencioso de registros inválidos (devuelve ‘null’ en el processor)
- SkipLimit: 500 registros tolerados por ejecución
- Listeners personalizados: CustomRetryListener, CustomSkipListener, JobCompletionListener

Configuración de paralelismo:

El TaskExecutor se configura mediante la propiedad ‘batch.parallelism’

```
batch:
  parallelism: ${BATCH_PARALLELISM:4}
  estadoscuenta:
    gridsize: ${BATCH_ESTADOS_GRID:4}
```

El particionador RangoLineasPartitioner divide el archivo cuentas_anuales.csv (1000 líneas) en N rangos y los procesa concurrentemente.

5.2 Patrón Backend for Frontend

Objetivo: Implementar tres BFFs independientes, cada uno optimizado para su canal.

BFF Web (puerto 8080)

- Canal: navegador de escritorio
- Campos por transacción: 6 (id, tipo, monto, fecha, anomalia, fechaProcesamiento)
- Optimización: datos completos con toda la información disponible
- Autenticación: ROLE_WEB_USER con expiración de 60 minutos

BFF Móvil (puerto 8083)

- Canal: aplicación móvil
- Campos por transacción: 4 (id, tipo, monto, fecha)
- Optimización: elimina anomalia y fechaProcesamiento para reducir ancho de banda
- Límite de resultados: máximo 5 por consulta
- Endpoint: /api/mobile/transacciones/recientes?limit=5
- Autenticación: ROLE_MOBILE_USER con expiración de 30 minutos

BFF ATM (puerto 8084)

- Canal: cajero automático
- Campos por saldo: 2 (cuentaId, saldo)
- Operaciones: consulta de saldo + retiro con actualización
- Timeouts: 2s conexión, 3s lectura (respuestas rápidas críticas)
- Autenticación: ROLE_ATM_USER con expiración de 5 minutos

Implementación técnica:

Cada BFF

1. Registra en Eureka como cliente de descubrimiento
2. Consume los microservicios vía RestClient con propagación del JWT
3. Aplica transformaciones específicas del canal
4. Implementa Circuit Breaker con fallback

5.3 Microservicios con Spring Cloud

Microservicio de Transacciones (8081)

- Responsabilidad: gestión de transacciones bancarias
- Persistencia: H2 inmemory con Spring Data JPA
- Kafka: productor en transaccionestopic (3 particiones)
- Endpoints:
  - GET /api/transacciones — lista todas
  - POST /api/transacciones — crea y publica evento

Microservicio de Cuentas (8082)

- Responsabilidad: consulta y retiro de cuentas
- Kafka: consumidor de transaccionestopic (grupo bancoxyzgroupcuentasv2)
- Endpoints:
  - GET /api/cuentas — lista
  - GET /api/cuentas/{numero} — detalle
  - POST /api/cuentas/{numero}/retiro — retiro con actualización

Microservicio de Pagos (8085)

- Responsabilidad: transferencias y depósitos
- Kafka: productor en pagostopic (3 particiones)

- Endpoints:
  - POST /api/pagos/transferencia
  - POST /api/pagos/deposito
  - GET /api/pagos — lista

Microservicio de Clientes (8086)

- Responsabilidad: CRUD de clientes
- Kafka: no utiliza
- Endpoints:
  - POST /api/clientes — crear
  - GET /api/clientes/{id} — consultar
  - GET /api/clientes/rut/{rut} — consultar por RUT
  - PUT /api/clientes/{id} — actualizar
  - DELETE /api/clientes/{id} — eliminar (soft delete)

5.4 Seguridad Distribuida

OAuth2 Authorization Server (9000)

- Implementado con Spring Authorization Server
- Cliente registrado: bffclient con secreto secret
- Grant type: client_credentials
- Tokens JWT firmados con RSA de 2048 bits

Validación de tokens:

- Todos los microservicios y BFFs actúan como Resource Servers
- Configuración mediante spring.security.oauth2.resourceserver.jwt.issueruri
- Propagación del token entre BFF y microservicios vía RestClient

Roles por canal:

- ROLE_WEB_USER — 1 hora
- ROLE_MOBILE_USER — 30 minutos
- ROLE_ATM_USER — 5 minutos

5.5 Mensajería Asíncrona

Apache Kafka en AWS EC2:

- Instancia t3.medium (2 vCPU, 4 GB RAM)
- AMI: Amazon Linux 2023
- Docker 25 + Docker Compose v2
- Kafka 3.7 (Confluent Platform 7.5.0)
- Zookeeper como coordinador

Configuración de listeners:

KAFKA_LISTENERS: INTERNAL://0.0.0.0:29092,EXTERNAL://0.0.0.0:9092

KAFKA_ADVERTISED_LISTENERS:

INTERNAL://kafka:29092,EXTERNAL://<IP_PUBLICA>:9092

KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL

Topics creados:

- transaccionestopic — 3 particiones, replicationfactor 1
- pagostopic — 3 particiones, replicationfactor 1

Flujo asíncrono:

1. Microservicio publica evento JSON en Kafka (producer)
2. Kafka persiste el mensaje en la partición correspondiente
3. Consumidores independientes leen desde su offset
4. El productor no espera confirmación (fully async)

5.6 Despliegue en la Nube

Recursos AWS provisionados:

- Instancia EC2: t3.medium en useast1
- Security Group: kafkabancoxyzsg con reglas:
    - Puerto 22 (SSH) — solo desde IP del administrador
    - Puerto 9092 (Kafka) — acceso público
    - Puerto 2181 (Zookeeper) — solo desde IP del administrador
    - Puerto 29092 (interno) — solo dentro de la VPC

Conexión local → nube:
Los microservicios locales fueron reconfigurados para apuntar al broker remoto:
SPRING_KAFKA_BOOTSTRAP_SERVERS: <IP_PUBLICA_EC2>:9092

Prueba endtoend:

1. Publicar pago desde micro local
2. Consumir mensaje desde EC2
3. Verificar offset en la partición

6. Resultados Obtenidos

6.1 Procesos Batch — Comparación de Paralelismo

Se ejecutó el job generacionEstadosCuentaAnualesJob con 4 configuraciones diferentes:

Tabla 1: Comparación de Paralelismo

Hilos	Particiones	Duración (ms)	Speedup	Eficiencia
1	1	2151	1.00x	100%
2	2	1432	1.50x	75%
4	4	1256	1.71x	43%
8	8	1282	1.68x	21%

Análisis:

- El speedup máximo se alcanza con 4 hilos (1.71x)
- Con 8 hilos el rendimiento se degrada ligeramente (+26ms vs 4 hilos)
- La eficiencia cae a partir de 4 hilos por overhead de contexto

Configuración óptima seleccionada: 4 hilos

Conteos de procesamiento (con 8 hilos):

- Registros leídos: 1000
- Registros escritos: 686
- Registros filtrados: 314 (31.4% del total)
- Cada partición procesó exactamente 125 líneas

6.2 BFFs — Optimización por Canal

Tabla 2: Comparación de respuestas por canal

Aspecto	BFF Web	BFF Móvil	BFF ATM
Campos por transacción	6	4	N/A
Campos por saldo	6	2	2
Operaciones	Consulta	Consulta	Consulta + Retiro
Timeout	5s	3s	3s
Rol JWT	ROLE_WEB_USER	ROLE_MOBILE_USER	ROLE_ATM_USER
Expiración token	60 min	30 min	5 min

Ejemplo de respuesta BFF Web (6 campos):

```
{
  "id": 1,
  "tipo": "DEPOSITO",
  "monto": 3000.0,
  "fecha": "20240630",
  "anomalia": false,
  "fechaProcesamiento": "20261011"
}
```

Ejemplo de respuesta BFF Móvil (4 campos):

```
{
  "id": 1,
  "tipo": "DEPOSITO",
  "monto": 3000.0,
  "fecha": "20240630"
}
```

Ejemplo de respuesta BFF ATM (2 campos):

```
{
  "cuentaId": "1000102",
  "saldo": 1000000.0
}
```

Reducción de payload: de 6 campos (Web) a 2 campos (ATM) = 67% menos datos por respuesta.

6.3 Resilience4j — Circuit Breaker

Prueba 1: Estado inicial (CLOSED)
- Estado: CLOSED
- bufferedCalls: 0
- failedCalls: 0

Prueba 2: Con microservicio detenido (5 llamadas)
- Todas devolvieron HTTP 200 con respuesta degradada
- Tiempos: 3.1s → 3.0s → 0.02s → 0.01s → 0.01s
- El fallback se activó correctamente

Prueba 3: Estado tras los fallos (OPEN)
- Estado: OPEN
- failureRate: 60%
- bufferedCalls: 5
- failedCalls: 3
- notPermittedCalls: 2

Prueba 4: Recuperación tras reiniciar el microservicio
- Estado final: CLOSED
- Recuperación automática en ~15 segundos

El Circuit Breaker funciona correctamente y devuelve respuestas degradadas en lugar de propagar errores 500.

6.4 Mensajería Kafka

Topics creados:
- transaccionestopic (3 particiones)
- pagostopic (3 particiones)

Prueba de publicación:
[PAGOSKAFKA] Evento publicado en pagostopic: id=1 monto=55555.0

Prueba de consumo desde AWS EC2:
{"id":1,"cuentaOrigen":"1000101","cuentaDestino":"1000102",
"monto":55555.0,"tipo":"TRANSFERENCIA","estado":"COMPLETADO",
"descripcion":"Evidencia Final AWS v2"}

Offsets verificados:
pagostopic:0:1
pagostopic:1:0
pagostopic:2:0

La comunicación asíncrona funciona correctamente entre el microservicio local y el broker en AWS.

6.5 Despliegue en AWS

Instancia EC2:

- ID: i0764f08defe23fbdb (sesión 1) / nueva instancia (sesión 2)
- Tipo: t3.medium
- Región: useast1
- Estado: running

Contenedores desplegados:

- kafka (confluentinc/cpkafka:7.5.0)
- zookeeper (confluentinc/cpzookeeper:7.5.0)

Tiempo de despliegue completo: ~15 minutos

- Lanzamiento EC2: 2 min
- Instalación Docker: 3 min
- Despliegue Kafka: 10 min (incluye pull de imágenes)

7. Análisis Comparativo: Legacy vs Nueva Arquitectura

Tabla 3: Comparación de características

Aspecto	Sistema Legacy	Nueva Arquitectura
Lenguaje	COBOL + Shell	Java 17 + Spring Boot
Plataforma	Mainframe	Docker + AWS Cloud
Escalabilidad	Vertical limitada	Horizontal ilimitada
Tiempo de despliegue	Horas (paradas)	Minutos (rolling)
Tolerancia a fallos	Manual	Automática (Resilience4j)
Costo operativo	Alto (mainframe)	Bajo (cloud payasyougo)
Velocidad de desarrollo	Lenta (talento escaso)	Rápida (Java estándar)
Monitoreo	Limitado	Actuator + Eureka
Integración	Protocolos propietarios	REST + Kafka
Seguridad	Perimetral	OAuth2 + JWT por servicio

Beneficios cuantificables:

- Batch: Reducción de tiempo de 2151ms a 1256ms con 4 hilos (43% más rápido)
- Payload móvil: Reducción de 6 a 4 campos (33% menos datos)
- Payload ATM: Reducción de 6 a 2 campos (67% menos datos)
- Resiliencia: Los errores 500 del legacy ahora son respuestas degradadas en milisegundos

8. Desafíos y Soluciones

Desafío 1: Configuración de Listeners de Kafka en AWS

Problema: Al desplegar Kafka en EC2, los clientes externos no podían conectarse porque el broker anunciaba su IP interna (kafka:29092) en lugar de la IP pública.

Solución: Configurar dos listeners diferenciados:
KAFKA_LISTENERS: INTERNAL://0.0.0.0:29092,EXTERNAL://0.0.0.0:9092
KAFKA_ADVERTISED_LISTENERS:
INTERNAL://kafka:29092,EXTERNAL://<IP_PUBLICA>:9092
KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL

Resultado: Los clientes externos se conectan a la IP pública:9092, mientras que los contenedores internos usan el listener interno.

Desafío 2: Firma del Fallback en Resilience4j

Problema: El Circuit Breaker devolvía errores 500 en lugar de ejecutar el fallback.

Causas identificadas:

- Firma del método fallback no coincidía con el método original
- Timeouts demasiado largos (10s conexión, 30s lectura) causaban que el cliente cortara antes
- minimumnumberofcalls con default de 100 impedía que el CB se abriera durante pruebas

Solución:

- Método fallback con firma idéntica + Throwable al final
- Timeouts reducidos: 2s conexión, 3s lectura
- minimumnumberofcalls: 3 para apertura rápida

Resultado: El fallback se ejecuta y devuelve respuestas 200 con datos degradados.

Desafío 3: Datos Sucios en CSV Legacy

Problema: El dataset legacy contenía ~30% de registros inválidos:

- Fechas en 4 formatos distintos (yyyyMMdd, ddMMyyyy, yyyy/MM/dd, dd/MM/yyyy)
- Fechas imposibles (20241301)
- Montos nulos, negativos o cero
- Tipos desconocidos (invalid, desconocido, unknown)

Solución:

- Validadores que prueban múltiples formatos de fecha
- Procesadores que devuelven null en lugar de lanzar excepción (filtro silencioso)
- Skip policies con límite de 500 registros

Resultado: El job completa exitosamente, procesando 686 registros válidos y filtrando 314 inválidos.

Desafío 4: Consistencia de Datos en el Batch Paralelo

Problema: Al usar particionamiento, cada hilo necesitaba leer un rango específico de líneas del CSV, pero FlatFileItemReader por defecto no respeta particiones.

Solución:

- Reader con @StepScope que recibe partition.min y partition.max desde el ExecutionContext
- Uso de linesToSkip(min) + maxItemCount(max min + 1)
- saveState(false) para evitar conflictos entre particiones

Resultado: Cada partición procesa exactamente 125 líneas (con 8 particiones).

Desafío 5: Limitaciones de Memoria en el Entorno de Desarrollo

Problema: Al levantar los 14 contenedores en simultáneo, el sistema operativo del desarrollador se quedaba sin memoria y mataba Docker Desktop.

Solución: Estrategia de levantamiento por fases:

- Levantar solo los contenedores necesarios para cada prueba
- Bajar los contenedores no utilizados
- Usar docker system prune para liberar espacio
- Documentar pruebas independientes por componente

Resultado: Todas las capturas de evidencia fueron tomadas exitosamente.

Desafío 6: Migración a PostgreSQL para Batch

Problema: El batchservice usaba H2 en memoria, perdiendo datos entre ejecuciones.

Solución:

- Agregar servicio postgres al ecosistema
- Cambiar spring.batch.jdbc.initializeschema para PostgreSQL
- Eliminar @EnableBatchProcessing (Spring Boot 3.x lo configura automáticamente)
- Ejecutar los jobs como contenedores efímeros (docker compose run rm)

Resultado: Persistencia real en PostgreSQL con 11 tablas (6 de Spring Batch + 5 de dominio).

9. Conclusiones

El proceso de migración del sistema legacy del Banco XYZ hacia una arquitectura moderna de microservicios puede calificarse como viable y exitoso. Los tres procesos batch críticos (Reporte de Transacciones Diarias, Cálculo de Intereses Mensuales y Generación de Estados de Cuenta Anuales) fueron reescritos íntegramente en Spring Batch, produciendo resultados equivalentes a los del sistema original y añadiendo mejoras sustanciales en rendimiento, tolerancia a fallos y mantenibilidad. Los mecanismos de retry, skip policies y validación multi-formato permitieron procesar datasets con altos niveles de suciedad (aproximadamente un 31% de registros inválidos) sin comprometer la ejecución completa de los jobs.

La implementación del patrón Backend for Frontend demostró ser una decisión arquitectónica acertada para optimizar la experiencia del usuario en cada canal. La diferenciación deliberada de respuestas según el tipo de cliente permitió reducir el payload hasta en un 67% para cajeros automáticos, manteniendo la información completa en el canal web y optimizando el ancho de banda en el canal móvil. Esta segmentación no solo mejora la velocidad de respuesta percibida por el usuario final, sino que también facilita la evolución independiente de cada canal, permitiendo que los equipos de frontend y backend trabajen con mayor autonomía y velocidad de iteración.

La resiliencia se consolidó como un pilar fundamental en el diseño de sistemas distribuidos. La incorporación de Resilience4j a través de Circuit Breakers con fallback permitió manejar de forma controlada las caídas parciales de servicios, transformando lo que en el sistema legacy habría sido un error crítico (HTTP 500) en una respuesta degradada elegante (HTTP 200 con datos vacíos o nulos). Las pruebas realizadas demostraron que el sistema pasa de estado CLOSED a OPEN en cuestión de milisegundos al detectar fallos, y se recupera automáticamente al restablecerse el servicio subyacente, garantizando una experiencia de usuario consistente incluso ante incidentes de infraestructura.

La incorporación de Apache Kafka como sistema de mensajería asíncrona aportó un nivel adicional de desacoplamiento entre los componentes del sistema. Los productores de eventos (microservicios de transacciones y pagos) operan de forma completamente independiente de los consumidores (microservicio de cuentas), permitiendo que cada componente evolucione, escale y se despliegue según sus propias necesidades. El despliegue de Kafka en una instancia EC2 de AWS validó la viabilidad de operar infraestructura de mensajería en la nube, con un tiempo de aprovisionamiento total inferior a 15 minutos y una conexión end-to-end verificada entre microservicios locales y el broker remoto.

Los procesos batch demostraron capacidad real de escalabilidad horizontal mediante particionamiento paralelo. La ejecución del job de Generación de Estados de Cuenta Anuales con diferentes configuraciones de hilos (1, 2, 4 y 8) evidenció un speedup máximo de 1.71x con 4 hilos, alcanzando una duración de 1256 milisegundos frente a los 2151 milisegundos de la ejecución secuencial. Aunque a partir de 8 hilos se observó un rendimiento decreciente debido

al overhead de creación de threads y contención en la base de datos, el experimento permitió identificar empíricamente la configuración óptima para el volumen de datos manejado y sentó las bases para el escalamiento futuro con datasets de mayor tamaño.

Finalmente, la implementación de seguridad distribuida mediante OAuth2 y tokens JWT firmados con RSA demostró ser un mecanismo robusto para garantizar la autenticación entre servicios sin generar acoplamiento entre ellos. Cada microservicio y BFF actúa como Resource Server independiente, validando los tokens localmente y aplicando políticas de autorización específicas por canal. Aunque la complejidad inicial de configuración es mayor que la de un sistema monolítico, la separación de responsabilidades y la ausencia de estado compartido simplifican significativamente la escalabilidad y el mantenimiento a largo plazo.

10. Propuestas de Mejora

1. Migración a Kubernetes. Reemplazar Docker Compose por Kubernetes (EKS en AWS) para orquestación más robusta y autoscaling.
2. Implementación del patrón Saga. Para transacciones distribuidas entre microservicios, implementando compensación en caso de fallos.
3. Observabilidad completa. Integrar Prometheus + Grafana para métricas, ELK Stack para logs centralizados, y Jaeger para trazas distribuidas.
4. API Gateway. Añadir Spring Cloud Gateway como punto de entrada único, con rate limiting, autenticación centralizada y enrutamiento.
5. Migración a Kafka en AWS MSK. Reemplazar la instancia EC2 con Kafka autogestionado por Amazon MSK (managed service).
6. CI/CD automatizado. Implementar GitHub Actions para build, test y despliegue automático a AWS.
7. Tests automatizados. Aumentar la cobertura con JUnit 5, Testcontainers y pruebas de integración endtoend.
8. Persistencia distribuida. Migrar de H2 inmemory a PostgreSQL replicado para cada microservicio.
9. Cache distribuido. Integrar Redis para cachear consultas frecuentes y reducir carga en microservicios.
10. Seguridad de secretos. Migrar credenciales hardcodeadas a AWS Secrets Manager o HashiCorp Vault.

11. Bibliografía

- Spring Batch Reference Documentation. Pivotal Software. https://docs.spring.io/springbatch/docs/current/reference/html/
- Spring Cloud Documentation. VMware Tanzu. https://spring.io/projects/springcloud
- Resilience4j Documentation. https://resilience4j.readme.io/
- Apache Kafka Documentation. The Apache Software Foundation. https://kafka.apache.org/documentation/
- Richardson, C. (2018). Microservices Patterns: With examples in Java. Manning Publications.
- Newman, S. (2021). Building Microservices: Designing FineGrained Systems (2nd ed.). O'Reilly Media.
- Newman, S. (2015). Backends For Frontends Pattern. https://samnewman.io/patterns/architectural/bff/
- OAuth 2.0 Authorization Framework. RFC 6749. IETF. https://tools.ietf.org/html/rfc6749
- JSON Web Tokens (JWT). RFC 7519. IETF. https://tools.ietf.org/html/rfc7519
- Amazon EC2 User Guide. AWS Documentation. https://docs.aws.amazon.com/ec2/

12. Anexos

Anexo A — Capturas de Pantalla

Todas las capturas se encuentran en la carpeta `evidencias/` del repositorio, con nombres numerados que indican el orden de lectura.

A.1 — Identificación de Procesos Clave (Criterio 1)

Captura 01 — Estructura del proyecto
`01-estructura-proyecto.png`
Muestra la organización completa del repositorio `banco-xyz-cloud`, con los módulos independientes por responsabilidad: 3 servicios de infraestructura, 4 microservicios de negocio, 3 BFFs, 1 batch-service y los archivos de orquestación.

A.2 — Propuesta de Arquitectura (Criterio 2)

Captura 02 — Docker Compose completo
`02-docker-compose.png`
Evidencia la definición declarativa del ecosistema completo: red compartida `banco-net`, variables de entorno, dependencias entre servicios, healthchecks y puertos expuestos. Corresponde a la implementación de la arquitectura propuesta.

A.3 — Procesos Batch con Spring Batch (Criterio 3)

Captura 03 — Estructura del batch-service
`00-estructura-batch.png`
Lista completa de archivos Java del módulo batch, mostrando la separación de responsabilidades: configs de jobs, DTOs, processors, validadores, listeners, repositorios y particionador.

Captura 04 — JobEstadoCuentaConfig con particionamiento
`00-job-estado-cuenta-config.png`
Código de la configuración del job de estados de cuenta. Se observan las anotaciones `@StepScope`, el uso de `partitioner(...)`, `gridSize(...)` y `taskExecutor(...)` para habilitar el procesamiento paralelo.

Captura 05 — Partitioner personalizado
`01-partitioner-codigo.png`
Implementación de `RangoLineasPartitioner` que divide el CSV en N particiones por rango de líneas. Cuenta las líneas reales del archivo y genera un `ExecutionContext` por partición con los límites `partition.min` y `partition.max`.

Captura 06 — Comparación de paralelismo (1, 2, 4 y 8 hilos)
`01-paralelismo-completo.png`
Ejecución del `generacionEstadosCuentaAnualesJob` con 4 configuraciones diferentes. Se aprecia la creación del `TaskExecutor` con N hilos y la distribución del trabajo en N particiones. Los tiempos muestran el speedup obtenido: 2151 ms (1 hilo), 1432 ms (2 hilos), 1256 ms (4 hilos) y 1282 ms (8 hilos), con un óptimo en 4 hilos.

Captura 07 — Conteos por step en PostgreSQL
`02-conteos-postgres.png`
Consulta a la tabla `batch_step_execution` mostrando los conteos de `READ_COUNT`, `WRITE_COUNT`, `FILTER_COUNT` y `STATUS` por cada partición ejecutada. Evidencia que se procesaron las 1000 líneas del dataset, con 686 registros persistidos y 314 filtrados por validaciones.

Captura 08 — Jobs 1 y 2 completados
`03-jobs-1-2.png`
Ejecución exitosa del `reporteTransaccionesDiariasJob` y `calculoInteresesMensualesJob`, ambos con estado `COMPLETED`. Demuestra que los 3 jobs del sistema batch están operativos.

A.4 — Backend For Frontend (Criterio 4)

Captura 09 — BFF ATM: respuesta con 2 campos
`01-atm-2-campos.png`
Respuesta del endpoint `/api/atm/saldo/{cuentaId}` retornando únicamente `cuentaId` y `saldo`, optimizando al máximo la respuesta para cajeros automáticos.

Captura 10 — BFF Web: respuesta con 6 campos
`02-web-6-campos.png`
Respuesta del endpoint `/api/web/transacciones` retornando el objeto completo: `id`, `tipo`, `monto`, `fecha`, `anomalia` y `fechaProcesamiento`. Comparado con el BFF ATM, se evidencia la optimización por canal.

Captura 11 — Retiro ATM con validaciones
`03-atm-retiro-validacion.png`
Operación exclusiva del BFF ATM. Se observan dos casos:
- Retiro exitoso con actualización de saldo en tiempo real.
- Rechazo por saldo insuficiente con mensaje descriptivo y sin alterar el saldo.

A.5 — Microservicios y Tolerancia a Fallos (Criterio 5)

Captura 12 — BffAtmService con Circuit Breaker
`00-bffatm-codigo.png`
Código de la clase de servicio del BFF ATM mostrando las anotaciones `@CircuitBreaker(name = "atmBffCB", fallbackMethod = ...)` sobre los métodos `obtenerSaldo` y `procesarRetiro`, junto con sus respectivos métodos de fallback.

Captura 13 — Fallback con el microservicio caído
`01-fallback-5llamadas.png`
5 llamadas consecutivas al BFF ATM después de detener el `microservicio-cuentas`. Todas las respuestas son `HTTP 200` (no 500), con `saldo: null` en el body. Los tiempos decrecen drásticamente a partir de la 3ª llamada por la apertura del Circuit Breaker.

Captura 14 — Circuit Breaker en OPEN y logs de fallback
`02-cb-open-logs.png`
Estado del `atmBffCB` en `OPEN` con `failureRate` de 60% y 3 fallos de 5 llamadas. Se complementa con los logs de la aplicación mostrando los mensajes `*** FALLBACK SALDO ACTIVADO ***` y el tipo de excepción capturada.

A.6 — Despliegue en la Nube (Criterio 6)

Captura 15 — Infraestructura EC2
`01-ec2-infraestructura.png`
Estado de la instancia EC2 `t3.medium` en la región `us-east-1` con IP pública asignada, junto con los contenedores activos (Kafka y Zookeeper) y los topics creados en el broker remoto (`transacciones-topic` y `pagos-topic`).

Captura 16 — EC2 con Kafka y Zookeeper corriendo
`01-ec2-kafka.png`
Vista del `docker ps` ejecutado sobre la instancia EC2, mostrando los contenedores de Kafka y Zookeeper en estado `Up` con los puertos `9092` y `2181` expuestos a internet a través del Security Group.

Captura 17 — Flujo end-to-end local → AWS
`02-end-to-end.png`
Prueba de integración completa:
- El microservicio local `microservicio-pagos` está configurado con `SPRING_KAFKA_BOOTSTRAP_SERVERS=35.153.202.176:9092` apuntando a la instancia EC2.
- Se publica una transferencia bancaria desde el entorno local.
- El mensaje JSON correspondiente se consume desde la EC2 mediante `kafka-console-consumer`.
- Se verifica que el mensaje viaja por internet desde Docker Desktop hasta el broker en la nube y es correctamente persistido en el topic.

Anexo B — Repositorio de Código
El código fuente completo está disponible en:
https://github.com/Dr3sden/bancoxyzcloud
Estructura del repositorio:

- configserver/ — Configuración centralizada
- eurekaserver/ — Service Discovery
- authserver/ — OAuth2 Authorization Server
- microserviciotransacciones/ — Kafka Producer
- microserviciocuentas/ — Kafka Consumer
- microserviciopagos/ — Kafka Producer
- microservicioclientes/ — CRUD de clientes
- bffweb/ — BFF Web (6 campos)
- bffmovil/ — BFF Móvil (4 campos)
- bffatm/ — BFF ATM (2 campos + retiro)
- batchservice/ — Procesos batch
- configrepo/ — Configuraciones YAML

Anexo C — Documentación Complementaria

- README.md — Descripción general del proyecto
- instrucciones.md — Guía de ejecución y pruebas
- despliegue.md — Guía de despliegue en AWS
- dockercompose.yml — Orquestación de contenedores

Anexo D — Tablas Comparativas Adicionales
Tabla 4: Resumen de microservicios y responsabilidades

Microservicio	Puerto	Persistencia	Kafka	Endpoints principales
transacciones	8081	H2	Producer	GET/POST /api/transacciones
cuentas	8082	H2	Consumer	GET /api/cuentas, POST

				/api/cuentas/{id}/retiro
pagos	8085	H2	Producer	POST /api/pagos/transferencia, /deposito
clientes	8086	H2	—	CRUD /api/clientes

Tabla 5: Comparación de formatos CSV procesados

Archivo	Filas	Válidas	Filtradas	% Filtrado
transacciones.csv	1000	~684	~316	31.6%
intereses.csv	1000	~686	~314	31.4%
cuentas_anuales.csv	1000	686	314	31.4%

Tabla 6: Tiempos de arranque de servicios

Servicio	Tiempo de arranque
configserver	15s
eurekaserver	20s
authserver	25s
microserviciotransacciones	30s
microserviciocuentas	30s
microserviciopagos	25s
microservicioclientes	20s
bffweb	25s
bffmovil	25s
bffatm	25s
batchservice	15s
postgres	10s
kafka	60s
zookeeper	15s

Tiempo total de arranque completo: ~3 minutos


DuocUC®
ONLINE